Project 4.1 — TripCraft-Multi-Agent-Travel-Itinerary-Generator

A practice twin of Project 4 — ContentForge. Same skills — a Supervisor routing specialist agents with `Command`, a shared LangGraph state, and a hard-limited review loop — on a **new, real-world domain**. There are **no steps here on purpose**: read the story, build the whole thing yourself, and only open the hints or the full solution if you get stuck.

Type: Multi-agent (5 agents: 1 Supervisor + 4 specialists) · **Stack:** Python, LangGraph, `Command` routing, LangChain · **Level:** Upper-Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you're building

You hand a travel planner a destination — "Paris" — and later they hand back a short, **realistic, sourced, reviewed day-by-day itinerary**, ready to use. They didn't do it alone. Someone looked up the real attractions for that city, someone grouped them into sensible themed days, someone wrote the itinerary, someone checked it was realistic and actually grounded in the looked-up attractions and sent it back until it was good enough — and a manager kept the whole team on track.

That's **TripCraft**. A **Supervisor** (the manager) reads a shared state and decides who works next, routing with LangGraph's `Command`. Four specialists each have one narrow job and a prompt written only for that job:

- **Research** looks attractions up with a tool (from a small local file of real places, each with a `[source]` tag) — never from the model's memory.
- **Planner** (the analysis role) groups the looked-up attractions into 2-4 themed days, **without losing a single one**.
- **Writer** writes the day-by-day itinerary from the plan **only** — nothing it wasn't given.
- **Reviewer** scores the itinerary against written criteria (realistic pacing, grounded in sourced attractions, covers the main ones) and sends it back with feedback — up to a hard limit.

It's a straight, honest pipeline: a known destination becomes a reviewed itinerary; an **unknown** destination stops cleanly ("nothing is known about this place yet") instead of inventing attractions; and a Writer and Reviewer that never agree stop at `MAX_REVISIONS` with a clear "couldn't agree" report, never an endless loop.

Why this is a real-world project: this is exactly the shape of a real travel-planning assistant, and — more generally — of any pipeline that turns one input into a reviewed, sourced document through genuinely separate phases (research → organize → draft → review). Trip planners, report generators, proposal builders, briefing tools all have this shape. Build it once here and you can build any of them.

Why it matters that you build it yourself, from the story: Project 4 you built step by step, with each step deciding the shape for you. Here there are no steps — you decide the state fields, the routing rules, the prompts, and where the loop stops, from the requirements alone. That's the actual job, and the only way to know the concepts stuck.

What You're Building (the requirements)

Build a terminal app: type a destination, get back a reviewed itinerary (or an honest "nothing known"), plus a visible log of which agents ran and why the Supervisor routed each way.

- A **Supervisor** node that routes to the right specialist based on the shared state, using `Command` for every handoff, and logs a `{step, reason}` entry for every decision.
- **Shared state** (`TeamState`) with **one writer per field** — Research writes the notes, Planner the day plan, Writer the draft, Reviewer the feedback/score; only the Supervisor writes the routing log. No agent overwrites another's work.
- A **tool-based Research** step: look attractions up from a local `attractions.json` (a few destinations, each attraction ending with a `[source]` tag); an unknown destination returns nothing and stops the run.
- A **Reviewer** that returns a **checked** verdict (a score 1-10 + feedback), with the pass mark set **in code** (`PASS_SCORE`), that can reject the Writer's itinerary and send it back — with a **hard limit** (`MAX_REVISIONS`), ending with either an accepted itinerary or a clear "couldn't agree" report.
- A clear, on-purpose choice for what happens when a step finds nothing usable (the unknown-destination stop).
- The full LangGraph graph: five nodes, one edge (`START → supervisor`); every other move is a `Command(goto=...)` a node returns.

Inputs: a destination (free text). **Outputs:** a reviewed itinerary, or an honest stop, plus the routing log. **Constraints:** no two agents redundantly look the same thing up for the same sub-task; the revision loop must be bounded; the itinerary must keep the `[source]` tags.

A Real Example (to build and test against)

- `"Paris"` → research finds 5 sourced attractions → grouped into 2-3 themed days → a written itinerary → reviewed and accepted (or revised first). The itinerary keeps every `[source]` tag.

- `"Atlantis"` (or any place not in your file) → research finds nothing → the team stops with "nothing is known about this destination yet", never an invented itinerary.
- **A deliberately weak itinerary** (in a test) → the Reviewer rejects it; after `MAX_REVISIONS` with no agreement, a clear "couldn't agree" report, no endless loop.

Put a few real destinations in `attractions.json` — Paris, Tokyo, Rome — each with 4-5 attractions and a `[source]` tag, so you can check the itinerary never invented one.

How to Practice This

1. Read this story and the requirements. **Don't look at the solution yet.**
2. Plan it on paper first: draw the five nodes, and decide who reads and who writes each state field (just like Project 4's "Plan Before You Code" step).
3. Build it yourself, in whatever order makes sense to you — there are no steps dictating one. Prove the Writer alone, then the Writer/Reviewer loop, then add Research and Planner, then the Supervisor and graph — or your own order.
4. Test it on the real example above: a known destination, an unknown one, and a never-agrees case.
5. **Only if you get stuck**, open the hints; **only to compare or check**, open the complete solution.

Done When

- A known destination flows supervisor → research → planner → writer → reviewer; a rejected itinerary goes back to the Writer (bounded by `MAX_REVISIONS`); the accepted result comes back with a full routing log.
- An unknown destination stops cleanly, with no invented attractions.
- No two agents redundantly look the same thing up; the loop always ends (accepted, or a clear "couldn't agree").
- Your tests pass: the settings, the lookup tool, the routing rules (on hand-made states), and a live run of the whole team.
- You can defend "why 4 agents and not 1" for this problem — in your own words.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.